

RESUME-HELPER PROJECT REPORT

Project-Aligned Version Based on Source Code

Table of Contents

- 1. Introduction
 - 1.1 Project Profile
 - 1.2 Project Abstract
- 2. Analysis
 - 2.1 Requirement Analysis
 - 2.2 Current System
 - 2.3 Proposed System
 - 2.4 Feasibility Study
- 3. About Back-End
- 4. About Front-End
- 5. About Other Tools Used
- 6. Software Requirement & Specification
 - 6.1 Introduction
 - 6.2 Functional Requirement
 - 6.3 Hardware/Software Requirement
- 7. Data Dictionary
- 8. E-R Diagram
- 9. Data Flow Diagram
 - 9.1 Level 0 DFD
 - 9.2 Level 1 DFD
 - 9.3 Level 2 DFD
- 10. Screen Layouts
- 11. Source Code
- 12. Reference

1. Introduction

1.1 Project Profile

Project Title: Resume-Helper

Project Type: Web Application

Project Category: Resume Analysis and ATS Scoring System

Target Users: Students, freshers, job seekers, and recruiters who want to check resume quality against a job description.

Resume-Helper is a web-based application that allows users to upload a resume in PDF or DOCX format and analyze it against a job description. The system extracts key information from the resume, calculates an ATS-style score, identifies missing skills, and gives improvement suggestions. It also provides authentication, resume history, rename and delete operations, and password reset functionality.

1.2 Project Abstract

The Resume-Helper project is designed to assist users in evaluating resumes more effectively. In the traditional process, candidates manually compare their resume with job descriptions and often fail to identify missing skills or weak sections. The proposed system automates this process by parsing uploaded resumes, extracting structured data such as skills, education, and experience, and then matching that information with a job description to generate an ATS-style score.

The system helps users understand how well their resume fits a target role and highlights areas for improvement. It also maintains a resume history so previous uploads and analyses can be reviewed. This makes the application useful for placement preparation, internships, and job applications.

2. Analysis

2.1 Requirement Analysis

Requirement analysis helps identify what the system should do and what users expect from it.

User Requirements:

- User should be able to register and log in.
- User should be able to upload resume files in PDF or DOCX format.
- User should be able to provide a job description.

- User should be able to analyze the uploaded resume.
- User should be able to view ATS score, missing skills, suggestions, and score breakdown.
- User should be able to access resume history.
- User should be able to rename or delete stored resumes.
- User should be able to reset password if forgotten.

System Requirements:

- The system must validate authentication and protect private routes.
- The system must accept only supported file formats.
- The system must parse resume text and extract structured fields.
- The system must compute ATS score based on defined scoring logic.
- The system must store users, resumes, and analysis records in the database.
- The system must provide a responsive and user-friendly interface.

2.2 Current System

In the current manual approach, users compare resumes and job descriptions by themselves. This process is slow, inaccurate, and highly dependent on personal judgment. Many students and job seekers do not know whether their resumes match the required skills, keywords, or experience expected in a job post.

Limitations of the Current System:

- Manual resume checking is time-consuming.
- Users may overlook missing keywords and skills.
- No automatic score or quality feedback is available.
- There is no structured resume analysis history.
- Freshers may not know how to improve weak resumes.

2.3 Proposed System

The proposed system, Resume-Helper, provides automated resume analysis. Users upload a resume, enter a job description, and receive an ATS-style score. The system extracts skills, education, and experience from the resume and compares them with the job description. It then shows missing skills, suggestions, and a category-wise breakdown.

Advantages of the Proposed System:

- Automates resume evaluation.
- Helps identify missing skills and weak areas.
- Provides resume score and analysis breakdown.
- Stores previous resume records and results.
- Improves the resume revision process for users.

2.4 Feasibility Study

Technical Feasibility: The project is technically feasible because it is implemented using React, TypeScript, Node.js, Express, MongoDB, Mongoose, and resume parsing libraries.

Economic Feasibility: The project is economical because it uses open-source tools and standard development resources.

Operational Feasibility: The system is practical because users can easily interact with it through a web interface.

Schedule Feasibility: The project is suitable for academic development because the modules are clearly separated into frontend, backend, parsing, analysis, and authentication.

3. About Back-End

The backend is built using Node.js, Express, and TypeScript. It manages authentication, resume upload, resume parsing, ATS analysis, history retrieval, rename and delete operations, and password reset logic. MongoDB is used as the database, and Mongoose is used to define and manage schemas.

Main Back-End Modules:

- User authentication controller for register, login, forgot password, and reset password.
- Resume controller for upload, history listing, rename, and delete.
- Analysis controller for ATS scoring and retrieving latest analysis.
- Parser service for reading PDF and DOCX resumes.
- Scoring engine for keyword score, skill score, experience score, format score, section score, and readability score.

4. About Front-End

The frontend is built with React, TypeScript, Vite, React Router, and Axios. It provides screens for login, sign-up, forgot password, reset password, dashboard, and resume history. The dashboard allows users to upload resumes and analyze them with a job description. The history page displays saved resumes and their latest ATS results.

Main Front-End Features:

- Authentication UI for login and registration.
- Protected dashboard route for authorized users.
- Resume upload interface for PDF and DOCX files.
- Job description input area.
- Analysis result display including score ring, missing skills, suggestions, and breakdown.
- Resume history with rename and delete actions.

5. About Other Tools Used

- **MongoDB:** Database for storing users, resumes, and analysis records.
- **Mongoose:** ODM for MongoDB schema management.
- **JWT:** Used for authentication and secure route access.
- **Multer:** Used to upload resume files.
- **pdf-parse:** Used for extracting text from PDF resumes.
- **Mammoth:** Used for extracting text from DOCX resumes.
- **Nodemailer:** Used in the password reset email flow.
- **Axios:** Used on the frontend for API communication.
- **Vite:** Used for frontend development and build tooling.

6. Software Requirement & Specification

6.1 Introduction

Software Requirement Specification defines the expected behavior, functional needs, and technical environment of the Resume-Helper system. The system is developed to support resume upload, parsing, ATS-style evaluation, and secure user-specific history management.

6.2 Functional Requirement

- The system shall allow users to register and log in.
- The system shall allow users to request password reset and set a new password.
- The system shall allow authenticated users to upload resumes.

- The system shall parse uploaded resumes and extract structured data.
- The system shall accept a job description and compute ATS-style score.
- The system shall display score, label, missing skills, suggestions, and score breakdown.
- The system shall maintain resume history for each user.
- The system shall support rename and delete operations for saved resumes.

6.3 Hardware/Software Requirement

Hardware Requirements:

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Hard Disk: Minimum 20 GB free space
- Internet connection for local development and deployment

Software Requirements:

- Operating System: Windows, Linux, or macOS
- Frontend: React, TypeScript, Vite, Axios, React Router
- Backend: Node.js, Express, TypeScript
- Database: MongoDB
- Libraries: Mongoose, JWT, Multer, Mammoth, pdf-parse, Nodemailer
- Code Editor: Visual Studio Code or similar IDE
- Browser: Google Chrome, Firefox, or Edge

7. Data Dictionary

The earlier data dictionary was not fully aligned with this project. The actual system stores data mainly in three collections: User, Resume, and Analysis. The corrected data dictionary is given below.

User Table / Collection

Field Name	Data Type	Description
_id	ObjectId	Unique identifier for the user
username	String	Name of the user

email	String	Unique email address of the user
password	String	Encrypted password
passwordResetToken	String / Null	Hashed token used for password reset
passwordResetExpires	Date / Null	Expiration time for password reset token
role	String	User role such as user, recruiter, or admin
createdAt	Date	Record creation timestamp
updatedAt	Date	Record update timestamp

Resume Table / Collection

Field Name	Data Type	Description
_id	ObjectId	Unique identifier for the resume
user	ObjectId	Reference to the user who uploaded the resume
fileUrl	String	Stored file path or placeholder value
originalFileName	String	Original name of uploaded resume file
parsedData.skills	Array of String	Skills extracted from the resume
parsedData.education	Array of String	Education entries extracted from the resume
parsedData.experience	Array of Object	Experience entries extracted from the resume
parsedData.experience.company	String	Company name in an experience record

parsedData.experience.role	String	Role or job title in an experience record
parsedData.experience.duration	Number	Experience duration in months
parsedData.experience.description	String	Description of work experience
parsedData.projects	Array of String	Project list; currently stored as empty array by parser
parsedData.rawText	String	Extracted plain text from uploaded resume
status	String	Resume state: uploaded, parsed, or analyzed
createdAt	Date	Record creation timestamp
updatedAt	Date	Record update timestamp

Analysis Table / Collection

Field Name	Data Type	Description
_id	ObjectId	Unique identifier for analysis record
resume	ObjectId	Reference to the analyzed resume
score	Number	Total ATS score between 0 and 100
label	String	Score label returned by the backend, such as Strong match
breakdown.keywordScore	Number	Score for keyword matching
breakdown.skillScore	Number	Score for skills matching
breakdown.experienceScore	Number	Score for experience relevance
breakdown.formatScore	Number	Score for resume format quality
breakdown.sectionScore	Number	Score for presence of important sections

breakdown.readabilityScore	Number	Score for readability
suggestions	Array of String	Improvement suggestions generated by scoring engine
jobDescription	String	Job description used for analysis
missingSkills	Array of String	Skills found in job description but missing in resume
createdAt	Date	Record creation timestamp
updatedAt	Date	Record update timestamp

8. E-R Diagram

The project is mainly centered on three entities: User, Resume, and Analysis.

```
User | |----< Resume | |----< Analysis
```

Relationship Explanation:

- One user can upload multiple resumes.
- One resume can have one or more analysis records over time.
- Analysis belongs to one resume, and resume belongs to one user.

9. Data Flow Diagram

9.1 Level 0 DFD

```
User ---> Resume-Helper System ---> ATS Analysis Result
```

9.2 Level 1 DFD

```
User | |--> Register / Login |--> Upload Resume |--> Enter Job Description
```

| Resume Parsing and Analysis System | |--> ATS Score |--> Missing Skills

--> Suggestions --> Resume History

9.3 Level 2 DFD

User | |--> Authentication Request | | | |--> Validate User | |--> Upload

Resume File | | | |--> Parse PDF / DOCX | |--> Extract Skills / Education /

Experience | |--> Save Resume Record | |--> Submit Job Description | |-->

Match Resume with JD |--> Calculate ATS Score |--> Generate Missing Skills

and Suggestions |--> Save Analysis Record |--> Display Result

10. Screen Layouts

- **Login Page:** Allows existing users to sign in using email and password.
- **Sign-Up Page:** Allows new users to create an account using username, email, and password.
- **Forgot Password Page:** Lets users generate a password reset request.
- **Reset Password Page:** Lets users set a new password using the reset token.
- **Dashboard Page:** Main page for uploading resumes and entering a job description for analysis.
- **Resume History Page:** Displays previously uploaded resumes and latest analysis results with rename and delete options.

11. Source Code

The project source code is divided into two major parts:

- **Frontend:** React application with pages for login, sign-up, dashboard, history, and password recovery.
- **Backend:** Express API with routes for authentication, resume upload, analysis, and history management.

Important backend files include server setup, controllers, models, parser service, and scoring engine. Important

frontend files include router setup, dashboard page, resume history page, auth context, and API service.

12. Reference

- [React documentation](#)
- [TypeScript documentation](#)
- [Node.js documentation](#)
- [Express.js documentation](#)
- [MongoDB documentation](#)
- [Mongoose documentation](#)
- [JWT documentation](#)
- [Multer documentation](#)
- [Mammoth documentation](#)
- [pdf-parse documentation](#)
- [Nodemailer documentation](#)